把广义表压进一条链

从 struct 定义、 BuildList 递归构造，到 IsEqual 判等

数据结构 · C 语言实现

路线

先讲结构，再讲构造，最后补判等

- 结构：一个 struct 同时容纳 data 与 sub_list
- 构造： BuildList 递归生成一条 GList
- 判等： IsEqual 同步比较两条链
- 缺口：题给代码没有 IsEqual 的函数体

tag 决定怎么读 union， next 决定往哪走

struct GListNode　　结点定义

```
1   typedef int ElemSet;
2 01 typedef enum { element, sublist } ElemTag;
3 02 typedef struct GListNode *Position;
4   typedef Position GList;
5
6   struct GListNode {
7       ElemTag tag;
8 03    union {
9           ElemSet data;
10          GList sub_list;
11      } u;
12 04   Position next;
13  };
```

结构 / 表示

读结构，先看三个词

tag、union、next 各管一件事。

01 **tag 是解释开关。**先看 element / sublist，再决定 union 里存了什么。

02 **Position 与 GList 都是指针。**表不是对象，表就是首结点的地址。

03 **union 共用一块内存。**同一结点要么是 data，要么是 sub_list，不会同时存在。

04 **next 只走同级。**向下进入子表，靠的是 sub_list。

记忆锚点：**tag 定语义，next 走平级，sub_list 下钻。**

子表不是特殊结构，只是 tag=sublist 的结点



例子 / 读图

跟着箭头读一遍

顶层与子表各是一条 next 链。

01 **L 只是首结点地址**，不是额外结点。

02 **顶层 next**：从 sublist 走到 data=30。

03 **sub_list 下钻**到 10 → 20 的子表链。

04 **同一结构重复使用**，没有第二种结点。

图上的 **next** 永远平走，**sub_list** 才换层。

BuildList 用临时头结点，把尾插写简单

BuildList　　递归构造

```
1   GList BuildList(int n) {
2       GList list, tail, tmp; int k, i;
3       list = (GList)malloc(sizeof(struct GListNode));
4 01    tail = list;                        /* 临时头结点 */
5       for (i = 0; i < n; i++) {
6           tmp = (GList)malloc(sizeof(struct GListNode));
7           scanf("%d", &k);
8 02        if (k == 0) {
9               tmp->tag = element; scanf("%d", &tmp->u.data);
10          } else {
11              tmp->tag = sublist;
12 03           tmp->u.sub_list = BuildList(k);
13          }
14 04       tail->next = tmp;
15          tmp->next = NULL; tail = tmp;
16      }
17 05   tmp = list; list = tmp->next; free(tmp);
18      return list;
19  }
```

构造 / BUILDLIST

四个动作，建完整条链

临时头结点、类型分派、递归、尾插。

01 **临时头结点。**让“第一个结点”和“后续结点”共用同一套尾插代码。

02 **k = 0 读 data。**这是原子结点；否则 k 就是子表的结点数。

03 **递归复用同一函数。**子表的 sub_list 仍是一条 GList。

04 **tail 尾插。**每次 O(1)，不用重复找链尾。

05 **归还临时头。**返回真正的首结点，不把哑结点带出去。

输入按“结点数 + 类型 + 数据”递归读取，**不是括号解析器。**

输入按结点数递归读取，不是括号串

记号	所在层	动作
2	main	顶层 n = 2
2	顶层	子表 n = 2
0 10	子表	element， data = 10
0 20	子表	element， data = 20
0 30	顶层	element， data = 30

主程序不碰结点，只做两次构建和一次判断

main 调用骨架

```
1   int main(void) {
2       GList glist1, glist2;
3       int n;
4
5       scanf("%d", &n);
6 01    glist1 = BuildList(n);
7       scanf("%d", &n);
8 02    glist2 = BuildList(n);
9
10 03   if (IsEqual(glist1, glist2) == true)
11          printf("true");
12      else
13          printf("false");
14
15      return 0;
16  }
```

主程序 / 调用

主程序不碰结点

只负责两次构建，把判等交给 IsEqual。

01 **读入规模。**n 直接作为 BuildList 的顶层结点数。

02 **重复一遍。**两条 GList 完全独立，结构可以不同。

03 **一次判断。**IsEqual 返回 bool，主程序只决定输出。

04 **但注意：**IsEqual 在题给代码中只有声明，函数体缺失（见下页）。

main 是编排者，不是算法本体。

IsEqual 只有声明，链接会失败

只声明、不定义：编译能过，链接会失败；补全函数体，程序才完整。

IsEqual 要同步走两条链，遇到 sublist 就递归

IsEqual　　补充实现

```
/* 补充实现：题给代码只有声明，没有函数体 */
bool IsEqual(GList a, GList b) {
    while (a != NULL && b != NULL) {
        if (a->tag != b->tag)
            return false;
        if (a->tag == element) {
            if (a->u.data != b->u.data)
                return false;
        } else {
            if (!IsEqual(a->u.sub_list, b->u.sub_list))
                return false;
        }
        a = a->next;
        b = b->next;
    }
    return a == NULL && b == NULL;
}
```

判等 / 补充实现

同步走，遇子表就递归

这段是为讲解补全的版本，不是题给原码。

01 **同步推进。**a、b 每次一起走 next，任何一方先空都要停下。

02 **先比结构。**tag 不同直接 false；都是 data 再比值。

03 **sublist 递归。**进入 u.sub_list，比较完再回到同级。

04 **收尾。**只有两边同时为空，整条链才相等。

时间 **O(结点总数)**，递归空间 **O(嵌套深度)**。

结构很省，但边界和资源管理要自己兜底

- 0 既表示原子，又让空子表无法编码
- malloc 与 scanf 未检查，异常输入会出错
- 只建链不释放，结束前应 FreeList
- 递归深度等于嵌套层数，深表会压栈
- 构建与判等均为 O(结点总数)

补上 IsEqual，程序才完整

补全函数 → 编译 → 相等得 true，改一个 data 得 false。